Python

Ingyu Bahng

May 2026

Contents

1 Introduction to Python

Core Concept 1.1: Python 3.0

Following several major releases over preceding decades (*Python 0.9.0 in February 1991, Python 1.0 in January 1994, Python 2.0 in October 2000*), Python 3.0 was released in December 2008 and has since evolved through incremental updates to Python 3.14.5 as of May 10, 2026.

The largest and most impactful version change came with the **transition from Python 2 to Python 3**, requiring over a decade for a complete migration due to **deliberate backward incompatibility**. This breaking change made many developers hesitant to migrate given the substantial time and cost involved in a complete update to Python 3 syntax and logic in existing codebases.

Today, Python 3 represents the definitive global industry standard for development, while Python 2 is structurally obsolete. Python 2 officially reached its End-of-Life (EOL) on January 1, 2020, and no longer receives security patches, bug fixes, or official maintenance.

Core Concept 1.2: CPython vs Jython vs PyPy

The original Python interpreter was implemented in **C**, a lower-level language positioned above **assembly** and **machine code**. This C-based implementation, known as **CPython**, remains the standard reference version distributed via python.org.

Alternative implementations include **Jython**, built on Java for seamless Java ecosystem integration, and **PyPy**, which offers significantly faster execution for pure Python code. However, Jython remains limited to Python 2.7 compatibility, while PyPy exhibits reduced compatibility with C-based libraries (*Core Concept 12.1*), many of which are essential to major projects.

2 Data Types & Structures

2.1 Singular Data Types

Core Concept 2.1: Numerics

Numeric data types encompass the set of data representations used to encode quantitative values within Python. Python provides several built-in numeric forms, each made for different purposes.

```
>>> a,b,c,d = 10, 0b1010, 0o12, 0x0a
>>> print(a, b, c, d)
10 10 10 10
>>> print(type(a), type(b), type(c), type(d))
<class 'int'> <class 'int'> <class 'int'> <class 'int'>

>>> print(type(3.14))
<class 'float'>

>>> print(type(2 + 5j))
<class 'complex'>
```

Code 1: Numeric data types `int`, `float`, and `complex`

(a) `int`: Integers represent whole numbers without a fractional component. They may be positive, negative, or zero. For example, the values `7`, `-42`, and `0` are all classified as integers in Python.

Integers can also be represented in **binary** (*base-2*), **octal** (*base-8*), or **hexadecimal** (*base-16*) notation using the prefixes `0b`, `0o`, and `0x`, respectively. For instance, `0b1010`, `0o12`, and `0x0a` all correspond to the decimal integer value `10`.

(b) `float`: Floats represent real numbers containing decimal components. For instance, `3.14` and `-0.001` are floating-point values commonly used to represent continuous numerical quantities.

(c) `complex`: Complex numbers represent values composed of both real and imaginary components in the form `a + bi`, where `i` denotes the imaginary unit satisfying $i^2 = -1$. As an example, the expression `2 + 5j` represents a complex number, where `j` denotes the imaginary component.

Core Concept 2.2: Boolean

`bool` represents logical truth values consisting exclusively of `True` or `False`. They are primarily used in conditional expressions, logical operations, and program control flow. For example, the expression `5 > 3` evaluates to `True`, whereas `5 < 3` evaluates to `False`.

```
>>> print(type(True), type(False))
<class 'bool'> <class 'bool'>
```

Code 2: `bool` data type

Core Concept 2.3: None

`NoneType` data type represents the **absence** of a value or the **intentional lack** of an assigned object. Python provides the singleton value `None` to denote null or undefined states within a program.

```
>>> print(type(None))
<class 'NoneType'>
```

Code 3: `None` data type

Core Concept 2.4: String

The `str` data type is an ordered sequence of textual characters surrounded by quotation marks, serving as the fundamental data type for text. For example, `"hello world"` constitutes a string literal. String objects support a variety of methods and operations for transformation and analysis.

```
>>> print(type("hello world"))
<class 'str'>
```

Code 4: `str` data types

(a) **Indexing and Slicing**: Accesses individual characters at specified positions using zero-based numerical indices, or extracts contiguous subsequences by specifying start and end positions.

```
>>> example_string = "hello world"
>>> print(example_string[0])
h
>>> print(example_string[3:9])
lo wor
```

Code 5: String indexing and slicing

(b) **Repetition**: Creates a new string by concatenating multiple copies of the original string.

```
>>> example_string = "hello world"
>>> print(example_string*2)
hello worldhello world
```

Code 6: String repetition

(c) **Length**: `len()` returns the total number of characters contained within the string.

```
>>> example_string = "hello world"
>>> print(len(example_string))
11
```

Code 7: Retrieving string length through `len()`

(d) **Strip**: `.strip()` removes leading and trailing characters from the string. By default, `.strip()` removes whitespace.

```
>>> example_string = "   hello world   "
>>> print(example_string)
   hello world
>>> stripped_string = example_string.strip()
>>> print(stripped_string)
hello world
```

Code 8: Removing leading and trailing whitespace from a string with `.strip()`

(e) **Find**: `.find()` locates the first occurrence of a specified substring and returns its starting index position.

```
>>> example_string = "hello world"
>>> substring = "l"
>>> example_string.find(substring)
2
```

Code 9: Finding a substring within a string through `.find()`

(f) **Count**: `.count()` returns the number of non-overlapping occurrences of a specified substring within the string.

```
>>> example_string = "hello world"
>>> substring = "l"
>>> example_string.count(substring)
3
```

Code 10: Determining a substring count within a string using `.count()`

(g) **Upper/Lower/Title**: `.upper()`, `.lower()`, and `.title()` converts all characters to uppercase, lowercase, or title case (*capitalizing the first letter of each word*), respectively.

```
>>> example_string = "hello world"
>>> print(example_string.upper())
HELLO WORLD
>>> print(example_string.lower())
hello world
>>> print(example_string.title())
Hello World
```

Code 11: Converting a string to upper, lower, and title case using `.upper()`, `.lower()`, and `.title()`

(h) **Escape Characters**: Special character sequences beginning with a backslash that represent non-printable or reserved characters. For instance, `\n` for newline, `\t` for tab, `\\` for the \character, and `\"` for the " character.

```
>>> example_string = "hello world\n\nhello world\\ \t \""
>>> print(example_string)
hello world

hello world\          "
```

Code 12: Special escape characters

(i) **Split**: `.split()` divides the string into a list of substrings based on a specified delimiter character or whitespace.

```
>>> example_string = "hello world this is an example string"
>>> print(example_string.split())
['hello', 'world', 'this', 'is', 'an', 'example', 'string']
```

Code 13: Spliting a string into components using `.split()`

2.2 Sequence Data Types

Core Concept 2.5: Lists

A `list` is an ordered, **mutable** (*modifiable after creation*) sequence that can contain elements of any data type. Lists are defined using square brackets and support dynamic modification of their contents. For example, `[1, 2, 3, "hello"]` constitutes a list literal containing both integers and a string.

(a) **Indexing and Slicing**: Accesses individual elements or subsequences using zero-based indices, analogous to string operations.

```
>>> example_list = [1, 2, 3, 4, 5]
>>> print(example_list[0])
1
>>> print(example_list[1:4])
[2, 3, 4]
```

Code 14: List indexing and slicing

(b) **Element Addition & Removal**: `.append()` and `.insert()` both add a single element to the end of

the list, modifying the list in place, but `.insert()` allows the addition to occur at a specified index position, shifting subsequent elements rightward. Conversely, `.remove()` deletes the first occurrence of a specified value from the list.

```
>>> example_list = [1, 2, 3]
>>> example_list.append(4) # append
>>> print(example_list)
[1, 2, 3, 4]
>>> example_list.insert(2, 5) # inserting 3 at index 2
>>> print(example_list)
[1, 2, 5, 3, 4]
>>> example_list.remove(2) # removal
>>> print(example_list)
[1, 5, 3, 4]
```

Code 15: List element addition through `.append()` / `.insert()` and removal through `.remove()`

(c) **Sort**: `.sort()` Arranges list elements in a specified order, modifying the list in place. By default, `.sort()` arranges the list in ascending order.

```
>>> example_list = [3, 1, 4, 1, 5, 9]
>>> example_list.sort() # ascending order
>>> print(example_list)
[1, 1, 3, 4, 5, 9]
```

Code 16: List sorting using `.sort()`

(d) **Join**: `.join()` concatenates list elements into a single **string** using a specified delimiter.

```
>>> example_list = ["hello", "world", "example"]
>>> print(" ".join(example_list))
hello world example
```

Code 17: Joining a list of strings using `.join()`

(e) **List Comprehension**: Constructs new lists using concise inline syntax that applies an expression to each element of an iterable (*Core Concept 3.2*).

```
>>> numbers = [1, 2, 3, 4, 5]
>>> print([x*2 for x in numbers])
[2, 4, 6, 8, 10]
```

Code 18: List comprehension

Core Concept 2.6: Tuples

A `tuple` is an ordered, **immutable** (*cannot be modified after creation*) sequence that can contain elements of any data type, making it suitable for fixed data structures that should not be modified. For example, `(1, 2, 3, "hello")` constitutes a tuple literal.

(a) **Indexing and Slicing**: Accesses individual elements or subsequences using zero-based indices, identical to list operations.

```
>>> example_tuple = (1, 2, 3, 4, 5)
>>> print(example_tuple[0], example_tuple[1:4])
1 (2, 3, 4)
```

Code 19: Tuple indexing and slicing

(b) **Tuple Packing and Unpacking**: Assigns multiple values simultaneously or extracts tuple elements into separate variables.

```
>>> x,y = (10, 20)
>>> print(x)
10
```

Code 20: Tuple packing and unpacking

Core Concept 2.7: Sets

A `set` is an **unordered** collection of **unique** elements that support mathematical set operations. Defined using curly braces or the `set()` constructor, sets automatically **eliminate duplicate values**. For example, `{1, 2, 3}` constitutes a set literal.

(a) **Element Addition & Removal**: `.add()` and `.update()` inserts a single element or multiple elements, respectively into the set. Conversely, `.remove()` deletes a specified element, raising an error if the element does not exist.

```
>>> example_set = {1, 2, 3}

>>> example_set.add(4) # singular addition
>>> print(example_set)
{1, 2, 3, 4}

>>> example_set.update([3, 4, 5]) # multiple addition
>>> print(example_set)
{1, 2, 3, 4, 5}

>>> example_set.remove(3) # removal
>>> print(example_set)
{1, 2, 4, 5}
```

Code 21: Set element addition through `.add()` / `.update()` and removal through `.remove()`

(b) **Set Operations**: Performs mathematical set operations including union, intersection, and difference.

```
>>> set_a = {1, 2, 3}
>>> set_b = {3, 4, 5}
>>> print(set_a.union(set_b))
{1, 2, 3, 4, 5}
>>> print(set_a.intersection(set_b))
{3}
```

Code 22: Set operations; `.union()` and `.intersection()`

(c) **Frozenset**: `frozenset()` creates an **immutable** version of a set that cannot be modified after creation.

```
>>> fset = frozenset({1,2,3})
>>> print(type(fset))
<class 'frozenset'>
```

Code 23: Frozenset using `frozenset()`

Core Concept 2.8: Dictionaries

A `dict` is an unordered collection of **key-value pairs** that enable efficient data lookup by key. Defined using curly braces with colon-separated pairs, dictionaries map unique keys to associated values. For example, `{"name": "Alice", "age": 30}` constitutes a dictionary literal.

(a) **Accessing Values**: Retrieves the value associated with a specified key using bracket notation.

```
>>> example_dict = {"name": "Alice", "age": 30}
>>> print(example_dict["name"])
Alice
```

Code 24: Accessing dictionary values

(b) **Adding and Modifying**: Assigns new key-value pairs or updates existing values.

```
>>> example_dict = {"name": "Alice"}
>>> example_dict["age"] = 30
>>> print(example_dict)
{'name': 'Alice', 'age': 30}
```

Code 25: Dictionary item addition and modification

(c) **Retrieving Dictionary Elements**: `.keys()`, `.values()`, and `.items()` returns a view object containing all dictionary keys, values, and key-value pairs, respectively.

```
>>> example_dict = {"name": "Alice", "age": 30}
>>> print(example_dict.keys(), example_dict.values())
dict_keys(['name', 'age']) dict_values(['Alice', 30])
>>> print(example_dict.items())
dict_items([('name', 'Alice'), ('age', 30)])
```

Code 26: Dictionary element retrieval using `.keys()`, `.values()`, and `.items()`

Core Concept 2.9: Ranges

A `range` is an memory-efficient immutable sequence of integers commonly used for loop iteration. It is generated using the `range()` function with `start`, `stop`, and optional `step` parameters.

The **basic range** creates a sequence of integers from 0 up to (*but not including*) a specified endpoint.

```
>>> print(list(range(5)))
[0, 1, 2, 3, 4]
```

Code 27: Basic range creation

However, you can also begin the range from a specified `start` value. Additionally, the `step` parameter allows for a specified increment between consecutive values.

```
>>> example_range = range(0, 10, 2)
>>> print(list(range(0, 10, 2)))
[0, 2, 4, 6, 8]
```

Code 28: Range with `start` and `step`

Core Concept 2.10: Bytes & Bytearrays

`bytes` and `bytearray` objects are sequences of integer values representing raw binary data. `bytes` objects are immutable, while `bytearrays` support modification. These types are essential for binary file operations, network protocols, and low-level data manipulation.

(a) **Bytes**: `b""` creates an immutable sequence of bytes from a string using specified encoding.

```
>>> example_bytes = b"hello"
>>> print(example_bytes, type(example_bytes))
b'hello' <class 'bytes'>
```

Code 29: Bytes using `b""` syntax

(b) **Bytearray**: `bytearray()` creates a mutable sequence of bytes that can be modified after creation.

```
>>> example_bytearray = bytearray(b"hello")
>>> example_bytearray[0] = 72 # the byte value for "H" is 72 in ASCII/UTF-8
>>> print(example_bytearray)
bytearray(b'Hello')
```

Code 30: Bytearray creation through `bytearray()`

(c) **Encoding and Decoding**: Converts between strings and bytes using character encodings.

```
>>> text = "hello"
>>> encoded = text.encode('utf-8')
>>> print(encoded)
b'hello'
>>> print(encoded.decode('utf-8'))
hello
```

Code 31: Encoding and decoding strings to bytes through `.encode()` and `.decode()`

2.3 Operators & Operands

Core Concept 2.11: Arithmetic Operators

Arithmetic operators perform mathematical calculations on numeric operands, returning numeric results.

(a) **Basic Arithmetic**: `+`, `-`, `*`, `/` perform addition, subtraction, multiplication, and division on numeric

operands, respectively. Division always returns a floating-point result. Note that `+` and `*` also operate on strings for concatenation and repetition (*Core Concept 2.4*).

```
>>> x,y = 10, 3
>>> print(x + y, x - y)
13 7
>>> print(x * y, x / y)
30 3.3333333333333335
```

Code 32: Basic arithmetic operators

(b) **Modulus**: `%` returns the remainder after division of the first operand by the second.

```
>>> x,y = 17, 5
>>> print(x % y)
2
```

Code 33: Modulus operator

(c) **Power**: `**` raises the first operand to the power of the second operand.

```
>>> x,y = 2, 3
>>> print(x ** y)
8
```

Code 34: Power operator

(d) **Floor Division**: `//` divides the first operand by the second and rounds down to the nearest integer.

```
>>> x,y = 17, 5
>>> print(x // y)
3
```

Code 35: Floor division operator

Core Concept 2.12: Assignment Operators

Assignment operators assign values to variables and can combine assignment with arithmetic operations for concise code.

(a) **Basic Assignment**: `=` assigns the value on the right to the variable on the left.

```
>>> x = 10
```

Code 36: Basic assignment operator

(b) **Compound Assignment Operators**: `+=`, `-=`, `*=`, `/=`, `%=`, `//=`, and `**=` combine each arithmetic operator (*Core Concept 2.11*) with assignment, performing the operation on the left operand with the right operand and assigning the result back to the left operand.

```
>>> x = 10
>>> x += 3 # addition
>>> print(x)
13
```

```
>>> x -= 2 # subtraction
>>> print(x)
11
>>> x *= 2 # multiplication
>>> print(x)
22
>>> x /= 4 # division
>>> print(x)
5.5
>>> # the same logic follow for modulus, floor division, and power
```

Code 37: Compound assignment operators

Core Concept 2.13: Comparison Operators

Comparison operators compare two operands and return a Boolean value (*Core Concept 2.2*) indicating the result of the comparison.

(a) **Equal To**: `==` returns `True` if both operands are equal in value.

```
>>> x,y = 5, 5
>>> print(x == y)
True
>>> print(x == 3)
False
```

Code 38: Equality comparison operator

(b) **Not Equal To**: `!=` returns `True` if operands are not equal in value.

```
>>> x,y = 5,3
>>> print(x != y)
True
>>> print(x != 5)
False
```

Code 39: Inequality comparison operator

(c) **Relational Comparisons**: `<`, `>`, `<=`, and `>=` compare the magnitude of two operands, returning `True` if the left operand is less than, greater than, less than or equal to, or greater than or equal to the right operand, respectively.

```
>>> x,y = 5,3
>>> print(x < y)
False
>>> print(x > y)
True
>>> print(x <= 5)
True
>>> print(x >= 5)
True
```

Code 40: Relational comparison operators

Core Concept 2.14: Logical Operators

Logical operators combine or modify Boolean expressions, returning Boolean values based on logical relationships.

(a) **And**: `and` returns `True` only if both operands are `True`.

```
>>> x,y = True, False
>>> print(x and x)
True
>>> print(x and y)
False
```

Code 41: `add` logical operator

(b) **Or**: `or` returns `True` if at least one operand is `True`.

```
>>> x,y = True, False
>>> print(x or y)
True
>>> print(y or y)
False
```

Code 42: `or` logical operator

(c) **Not**: `not` returns the opposite Boolean value of the operand.

```
>>> x,y = True, False
>>> print(not x, not y)
False True
```

Code 43: `not` logical operator

3 Control Flow

3.1 Conditionals

Core Concept 3.1: if, elif, else

Conditional statements enable selective code execution based on Boolean conditions. An `if` statement evaluates a specified condition and executes its code block only when the condition evaluates to `True`. The `elif` (*else-if*) clause allows sequential evaluation of multiple conditions, executing only when all preceding conditions evaluate to `False`. The `else` clause provides a default code block that executes when all preceding conditions fail.

```
>>> x, y = 10, 5
>>> if x > y:
...     print("x is greater than y") # conditional ends here
... elif y > x:
...     print("y is greater than x")
... else:
...     print("x is equal to y")
```

```
...
x is greater than y
```

Code 44: Conditional `if` / `elif` / `else` statement block

3.2 Loops

Core Concept 3.2: for, break, continue

The `for` loop enables iteration over elements in an iterable sequence (*lists, tuples, strings, ranges*). Each iteration assigns the current element to a loop variable and executes the indented code block. The loop terminates automatically upon processing all elements, or manually via the `break` statement, which immediately exits the loop regardless of whether all elements have been processed.

```
>>> for num in [1, 2, 3, 4]:
...     if num == 3:
...         break
...     print(num)
...
1
2
```

Code 45: `for` loop with `break`

Conversely, the `continue` statement skips the remainder of the current iteration and proceeds immediately to the next iteration without terminating the loop entirely.

```
>>> for num in [1, 2, 3]:
...     if num == 2:
...         continue
...     print(num)
...
1
3
```

Code 46: `for` loop with `continue`

Core Concept 3.3: while

The `while` loop repeatedly executes its code block as long as a specified Boolean condition remains `True`. Unlike `for` loops that iterate over finite sequences, `while` loops continue indefinitely until their condition evaluates to `False`, making them suitable for scenarios where the iteration count is not predetermined.

```
>>> x = 0
>>> while x < 3:
...     print(x)
...     x += 1
...
0
1
2
```

Code 47: Basic `while` loop

The `break` and `continue` statements (*Core Concept 3.2*) function identically within `while` loops, allowing for premature loop termination or iteration skipping, respectively.

```
>>> x = 0
>>> while x < 5:
...     x += 1
...     if x == 2:
...         continue
...     if x == 4:
...         break
...     print(x)
...
1
3
```

Code 48: `while` loop with `continue` and `break`

4 Functions

Core Concept 4.1: def

The `def` keyword defines a **function**—a reusable block of code that executes when called. Functions accept input through **parameters** (*variables defined in the function*), process data according to their internal logic, and optionally return output values using the `return` statement.

```
>>> def greet(name):
...     return f"Hello, {name}!"
...
>>> print(greet("Alice"))
Hello, Alice!
>>> print(greet("Bob"))
Hello, Bob!
```

Code 49: Basic function definition through `def`

Python supports **positional parameters** (*matched by position*) and **keyword parameters** (*matched by name*). Additionally, default parameter values allow functions to be called with fewer arguments than defined parameters, providing fallback values when arguments are omitted.

```
>>> def add(x, y):
...     return x + y
...
>>> print(add(3, 5))
8
>>> print(add(x=10, y=20))
30

>>> def greet(name, greeting="Hello"):
...     return f"{greeting}, {name}!"
...
>>> print(greet("Alice"))
Hello, Alice!
>>> print(greet("Bob", "Hi"))
```

```
Hi, Bob!
```

Code 50: `def` function positional and keyword arguments

`*args` and `**kwargs` enable functions to accept variable numbers of arguments. `*args` collects excess positional arguments into a tuple, while `**kwargs` collects excess keyword arguments into a dictionary.

```
>>> def print_args(*args):
...     for arg in args:
...         print(arg)
...
>>> print_args(1, 2, 3, 4, 5)
1
2
3
4
5

>>> def print_kwargs(**kwargs):
...     for key, value in kwargs.items():
...         print(f"{key}: {value}")
...
>>> print_kwargs(name="Alice", age=30, city="NYC")
name: Alice
age: 30
city: NYC
```

Code 51: `*args`; `def` function variable position arguments

Functions can utilize both `*args` and `**kwargs` simultaneously to achieve maximum flexibility in parameter handling.

```
>>> def flexible_func(x, y, *args, **kwargs):
...     print(f"x: {x}, y: {y}")
...     print(f"args: {args}")
...     print(f"kwargs: {kwargs}")
...
>>> flexible_func(1, 2, 3, 4, 5, name="Alice", age=30)
x: 1, y: 2
args: (3, 4, 5)
kwargs: {'name': 'Alice', 'age': 30}
```

Code 52: `**kwargs`; `def` function variable keyword arguments

Core Concept 4.2: lambda

The `lambda` keyword creates **anonymous functions**—small, unnamed functions defined in a single expression. Lambda functions are syntactically restricted to a single expression and implicitly return the result of that expression, making them suitable for simple operations that don't warrant full function definitions (*Core Concept 4.1*).

```
>>> def add(x, y): # regular function
...     return x + y
...
>>> add_lambda = lambda x, y: x + y # equivalent lambda function
```

```
>>> print(add(3, 5))
8
>>> print(add_lambda(3, 5))
8
```

Code 53: `lambda` function

Lambda functions are commonly used as **arguments to higher-order functions** that accept functions as parameters, such as `map()`, `filter()`, and `sorted()`.

```
>>> numbers = [1, 2, 3, 4, 5]

>>> squared = list(map(lambda x: x**2, numbers)) # map: apply function to each element
>>> print(squared)
[1, 4, 9, 16, 25]

>>> evens = list(filter(lambda x: x % 2 == 0, numbers)) # filter: keep elements where
 ↪ function returns True
>>> print(evens)
[2, 4]

>>> words = ["apple", "pie", "zoo", "a"] # sorted: custom sorting key
>>> sorted_by_length = sorted(words, key=lambda x: len(x))
>>> print(sorted_by_length)
['a', 'pie', 'zoo', 'apple']
```

Code 54: `lambda` with `map()` function

Lambda functions can **reference variables from their enclosing scope**, meaning that they have the ability to access and utilize variables that were defined outside of its own internal parameter list and thus creating closures that capture external states.

```
>>> def make_multiplier(n):
...     return lambda x: x * n
...
>>> times_two = make_multiplier(2)
>>> print(times_two(5))
10
```

Code 55: `lambda` function referencing variables from their enclosing scope

Core Concept 4.3: Decorators

Decorators are functions that **modify** or enhance the behavior of other functions without altering their source code. Decorators wrap the target function, adding functionality before or after its execution, and are applied using the `@` syntax.

```
>>> def uppercase_decorator(func):
...     def wrapper(*args, **kwargs):
...         result = func(*args, **kwargs)
...         return result.upper()
...     return wrapper
...
>>> @uppercase_decorator
... def greet(name):
```

```
...     return f"hello, {name}"
...
>>> print(greet("alice"))
HELLO, ALICE
```

Code 56: Basic uppercase decorator

Decorators can accept arguments by adding an additional layer of function nesting, enabling parameterized decoration behavior.

```
>>> def repeat(times):
...     def decorator(func):
...         def wrapper(*args, **kwargs):
...             for _ in range(times):
...                 result = func(*args, **kwargs)
...             return result
...         return wrapper
...     return decorator
...
>>> @repeat(times=2)
... def say_hello():
...     print("Hello!")
...
>>> say_hello()
Hello!
Hello!
```

Code 57: Parameterized multiplier decorator

Multiple decorators can be stacked on a single function, applying transformations in bottom-to-top order (*the decorator closest to the function executes first*).

```
>>> def bold(func):
...     def wrapper(*args, **kwargs):
...         return f"**{func(*args, **kwargs)}**"
...     return wrapper
...
>>> def italic(func):
...     def wrapper(*args, **kwargs):
...         return f"_{func(*args, **kwargs)}_"
...     return wrapper
...
>>> @bold
... @italic
... def text():
...     return "Hello"
...
>>> print(text())
**_Hello_**
```

Code 58: Stacked decorators

Core Concept 4.4: Generators

Generators are functions that use the `yield` keyword to produce a sequence of values lazily, generating each value **on-demand** rather than computing and storing all values in memory simultaneously. This enables efficient iteration over large or infinite sequences. Unlike regular functions that return once and terminate,

generators **maintain their state** between `yield` statements, resuming execution from where they left off when the next value is requested.

```
>>> def count_up_to(n):
...     count = 1
...     while count <= n:
...         yield count
...         count += 1
...
>>> count_gen = count_up_to(10)

>>> for _ in range(2):
...     print(next(count_gen))
...
1
2
>>> for _ in range(3): # starts from where it left off in the previous for loop
...     print(next(count_gen))
...
3
4
5
```

Code 59: Generator state maintenance using `next()`

Generator expressions provide a concise syntax for creating generators, analogous to list comprehensions but with parentheses instead of brackets, **producing values lazily without constructing the entire sequence in memory**. Once a generator yields a value, that value is immediately forgotten by the generator. Once you have looped through the entire generator, it is completely empty. In programming, this is known as **generator exhaustion**.

```
>>> squares_gen = (x**2 for x in range(3))

>>> print(list(squares_gen))  # first time: we consume the generator
[0, 1, 4]

>>> print(list(squares_gen))  # second time: the generator is already exhausted
[]
```

Code 60: Generator exhaustion

5 Object Oriented Programming

5.1 Classes

Core Concept 5.1: Classes, Attributes, and Methods

A `class` is a blueprint for creating objects that encapsulates data and behavior into a cohesive construct, enabling modular code organization and reusability.

```
>>> class BankAccount:
...     bank = "Chase" # class attribute
...     def __init__(self, owner, balance=0):
...         self.owner = owner # instance attribute
...         self.balance = balance # instance attribute
...     def deposit(self, amount): #method
```

```
...         self.balance += amount
...         return self.balance
...     def withdraw(self, amount):
...         if amount <= self.balance:
...             self.balance -= amount
...             return self.balance
...         return "Insufficient funds"
...
>>> account = BankAccount("Alice", 100)
>>> print(account.deposit(50))
150
>>> print(account.withdraw(30))
120
>>> print(account.balance)
120
```

Code 61: `class` instantiation, attributes, and methods

The `__init__()` method serves as a special constructor that initializes new class instances, executing automatically upon object instantiation. The `self` parameter references the specific instance being created or manipulated, providing access to that instance's attributes and methods.

Attributes represent the data stored within a class and are categorized into two distinct types.

- **Class attributes** are shared uniformly across all instances of a class. In the `BankAccount` example, `bank = "Chase"` constitutes a class attribute accessible to all `BankAccount` objects.
- **Instance attributes** are unique to each individual object. Attributes such as `self.owner` and `self.balance` must be explicitly provided during instantiation, as demonstrated by `account = BankAccount("Alice", 100)`.

Methods are functions defined within a class namespace that operate on instance data. These functions are bound to the class and accessible only through class instances. In the example above, `deposit()` and `withdraw()` exemplify methods that represent the behavioral logic of the `BankAccount` class, specifically the `self.balance` attribute.

Core Concept 5.2: Setters & Getters

Setters and **getters** are methods that control access to an object's attributes outside of the initialization of the class. These methods of control can be written in one of two ways.

(a) **Explicit Method**: The setter and getter are just written as plain functions.

```
>>> class Example:
...     def setName(self, n): # setter
...         self.name = n
...     def getName(self): # getter
...         return self.name
...
>>> p1 = Example()
>>> p1.setName("John")
>>> print(p1.getName())
John
```

Code 62: Explicit setter and getter method

(b) **Decorator Method**: The setter and getter methods are decorated with the `@property` and `@<attribute>.setter` annotations.

By convention, attributes prefixed with a single underscore in the format of `_attribute` indicate internal implementation details that should not be accessed directly, though Python does not enforce true privacy.

```
>>> class PythonicExample:
...     def __init__(self, name):
...         self._name = name  # Protected backing attribute
...     @property
...     def name(self):  # Getter decorator
...         return self._name
...     @name.setter
...     def name(self, n):  # Setter decorator
...         if not n:
...             raise ValueError("Name cannot be empty")
...         self._name = n
...
>>> p2 = PythonicExample("John")
>>> p2.name = "Sally"  # Intercepted by the setter seamlessly
>>> print(p2.name)     # Intercepted by the getter seamlessly
Sally
```

Code 63: Decorator setter and getter method

The difference is how Python handles attribute access under the hood. The explicit method relies on traditional function calls like `p1.setName()`, which creates a nightmare when trying to convert all `p1.name = <name>` lines for all class instances to the `p1.setName()` method, resulting in every file interacting with that variable having to be refactored.

Conversely, the decorator method hooks directly into Python's descriptor protocol such as `p2.name = "Sally"` (*setter*) and `p2.name` (*getter*) in the above example. This wraps the underlying functions into a property descriptor object, allowing outside code to maintain the clean, direct assignment syntax of `p2.name = "Sally"`.

The decorator method is simply the undisputed industry standard *way to handle setters and getters because it gives the developer the power to add data validation, transformation, or read-only access control in the background with ease. For instance, omitting the matching setter to a property getter creates a strict read-only functionality for an attribute.*

```
>>> class ReadOnlyExample:
...     def __init__(self, name):
...         self._name = name  # Protected backing attribute
...     @property
...     def name(self):  # Getter decorator
...         return self._name
...     # Note: The .setter decorator is intentionally omitted to make it read-only
...
>>> p3 = ReadOnlyExample("John")
>>> print(p3.name)     # Read access works perfectly via the getter
John

>>> # Attempting to modify the attribute will now throw an AttributeError
>>> p3.name = "Sally"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: property 'name' of 'ReadOnlyExample' object has no setter
```

Code 64: Setter omission creating a read-only functionality for an attribute

5.2 The Four Pillars of Object Oriented Programming

Core Concept 5.3: Encapsulation

Encapsulation is the practice of bundling data attributes and their associated methods into a cohesive class unit while restricting direct external access through access control mechanisms. This design principle protects internal state integrity by exposing controlled interfaces rather than allowing direct manipulation of object internals.

```
>>> class BankAccount:
...     def __init__(self, owner, initial_deposit):
...         self.owner = owner
...         self.__balance = initial_deposit
...     @property
...     def balance(self):
...         return f"{self.__balance:,.2f}"
...     def display_balance(self):
...         return f"{self.__balance:,.2f}"
...
>>> account = BankAccount("Ingyu", 1000)
>>> print(account.balance) # getter method
1,000.00
>>> print(account.display_balance()) # explicit function
1,000.00
>>> print(account._BankAccount__balance) # name mangling
1000

>>> print(account.__balance) # private attributes cannot be directly accessed
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'BankAccount' object has no attribute '__balance'
```

Code 65: Encapsulation and `class` private attributes using `__attribute` syntax

The double underscore prefix (`__attribute`) designates an attribute as private, triggering Python's name mangling mechanism that converts the attribute name to `_ClassName__attribute`. While this prevents direct external access via the original attribute name, the value remains retrievable through properly designed accessor methods (*getters*), explicit retrieval functions, or by explicitly invoking the mangled name syntax—though the latter circumvents the intended encapsulation and is considered poor practice.

Core Concept 5.4: Inheritance

Inheritance is a fundamental object-oriented mechanism whereby a child class (*subclass*) acquires the attributes and methods of a parent class (*superclass*), facilitating code reuse and enabling the hierarchical extension of functionality without redundancy.

```
>>> class Animal:
...     def __init__(self, name):
...         self.name = name
...     def eat(self):
...         return f"{self.name} is eating."
...     def speak(self):
```

```
...         return f"{self.name} makes a generic sound."
...
>>> class Dog(Animal):
...     def speak(self):
...         return f"{self.name} barks: Woof!"
...
>>> my_dog = Dog("Buddy")
>>> print(my_dog.eat())
Buddy is eating.
>>> print(my_dog.speak())
Buddy barks: Woof!
```

Code 66: Inheritance of parent `Animal` by child `Dog`

In the above code block, the `Dog` class inherits all attributes and methods from the `Animal` parent class. When a child class defines a method with an identical name to a parent class method—such as `speak()` in this example—the child implementation completely overrides the parent version, a behavior known as **method overriding**.

In addition to the simple inheritance shown above, the `super()` function allows the user to return a proxy object that delegates method calls to the parent class, enabling controlled inheritance and method extension. This mechanism serves two primary purposes.

- **Constructor Extension**: Child class constructors frequently require both parent initialization logic and child-specific setup. The expression `super().__init__()` delegates baseline initialization to the parent constructor, ensuring proper attribute inheritance while permitting the addition of child-specific attributes.
- **Method Extension**: While method overriding replaces parent functionality entirely, `super().method_name()` allows child classes to invoke parent method logic before or after executing child-specific operations, thereby extending rather than replacing inherited behavior.

```
>>> class Vehicle:
...     def __init__(self, brand, model):
...         self.brand = brand
...         self.model = model
...     def description(self):
...         return f"{self.brand} {self.model}"
...
>>> class Car(Vehicle):
...     def __init__(self, brand, model, doors):
...         super().__init__(brand, model)
...         self.doors = doors
...     def description(self):
...         base_info = super().description()
...         return f"{base_info} with {self.doors} doors"
...
>>> car = Car("Toyota", "Camry", 4)
>>> print(car.description())
Toyota Camry with 4 doors
```

Code 67: Method extension using `super()`

Core Concept 5.5: Polymorphism

Polymorphism is the capacity of different classes to implement identically named methods, enabling a single unified interface to operate on diverse object types while invoking class-specific behavior. This principle allows functions to process heterogeneous objects uniformly, with runtime behavior determined by each object's concrete class implementation.

```
>>> class MasterCard:
...     def process_payment(self, amount):
...         return f"Processing ${amount:.2f} via MasterCard networks."
...
>>> class PayPal:
...     def process_payment(self, amount):
...         return f"Routing ${amount:.2f} through PayPal wallets."
...
>>> class Bitcoin:
...     def process_payment(self, amount):
...         return f"Broadcasting ${amount:.2f} to blockchain."
...
>>> class ApplePay:
...     def process_payment(self, amount):
...         return f"Authenticating ${amount:.2f} via Apple."
...
>>> def checkout_gate(payment_processor, total):
...     print(payment_processor.process_payment(total))
...
>>> visa, wallet, crypto, mobile = MasterCard(), PayPal(), Bitcoin(), ApplePay()

>>> checkout_gate(visa, 150.00)
Processing $150.00 via MasterCard networks.
>>> checkout_gate(wallet, 150.00)
Routing $150.00 through PayPal wallets.
>>> checkout_gate(crypto, 150.00)
Broadcasting $150.00 to blockchain.
>>> checkout_gate(mobile, 150.00)
Authenticating $150.00 via Apple.
```

Code 68: Polymorphism and identically named method `process_payment` executing different actions based on paired `class`

Each payment processor class implements an identically named `process_payment()` method with class-specific logic. The `checkout_gate()` function accepts any payment processor object and invokes its `process_payment()` method, demonstrating polymorphic behavior: the same function call produces different outputs depending on the runtime type of the object passed as an argument. This design enables extensibility—new payment processor classes can be added without modifying the `checkout_gate()` function, provided they implement the `process_payment()` interface.

Core Concept 5.6: Abstraction

Abstraction is a fundamental architectural principle that conceals complex, low-level implementation details behind simplified interfaces, restricting external entities to interacting only with an object's external behavior rather than its inner mechanisms, a constraint often realized through **encapsulation** (*Core Concept 5.3*), **inheritance** (*Core Concept 5.4*), and **polymorphism** (*Core Concept 5.5*).

Explicit abstraction—facilitated by Python's `abc` library—enforces a mandatory structural baseline for all

derived subclasses. Furthermore, it prohibits direct instantiation of the abstract base class, ensuring access is only mediated through its child class implementations.

```
>>> from abc import ABC, abstractmethod
>>> class PrinterBlueprint(ABC):
...     @abstractmethod
...     def print_page(self, text):
...         pass
...
>>> class OfficePrinter(PrinterBlueprint): # Matches the blueprint EXACTLY
...     def print_page(self, text):
...         return f"Printing office document: {text}"
...
>>> class BrokenPrinter(PrinterBlueprint): # Flaw: accidental rename
...     def output_text(self, text):
...         return f"Printing: {text}"
...
>>> printer_1 = OfficePrinter() # success
>>> printer_2 = BrokenPrinter() # renamed mandatory method
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class BrokenPrinter with abstract method print_page
>>> printer_3 = PrinterBlueprint() # can't call the abstract parent class
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class PrinterBlueprint with abstract method
 ↪ print_page
```

Code 69: Explicit abstraction using the `abc` module

As demonstrated above, the instantiation of the `BrokenPrinter` class fails because it neglects to implement the required `print_page()` method prescribed by its abstract base class, `PrinterBlueprint`. Conversely, the `OfficePrinter` instantiation succeeds because it strictly adheres to the same method signature outlined within `PrinterBlueprint`.

6 Error Handling & Debugging

6.1 Exceptions Handling

Core Concept 6.1: Exceptions

Exceptions are runtime errors that cause the abnormal termination of a program and its resources. Examples include `ZeroDivisionError`, which occurs when attempting to divide a value by zero, and `IndexError`, which arises when accessing an element within a collection (*such as a list or array*) using an invalid index.

Core Concept 6.2: Custom Exceptions

Developers can define **custom exceptions** by subclassing the base `Exception` class, which can subsequently be triggered using the `raise` statement.

```
>>> class InvalidAgeError(Exception):
...     pass # no need for constructor
```

```
...
>>> def verify_age(age):
...     if age < 0:
...         raise InvalidAgeError(f"Age cannot be negative. Received: {age}")
...     print(f"Age {age} successfully verified.")
...
>>> try:
...     print("System: Verifying user age...")
...     verify_age(-5)
...
... except InvalidAgeError as e:
...     print(f"Error Type: {type(e).__name__}")
...     print(f"Error Message: {e}")
...
System: Verifying user age...
Error Type: InvalidAgeError
Error Message: Age cannot be negative. Received: -5
```

Code 70: Custom `InvalidAgeError` exception

As demonstrated above, the custom exception subclass does not require an explicit constructor. It inherits the constructor from the base `Exception` class, which is already designed to accept arguments—specifically, a string representing an error message.

Core Concept 6.3: try, except, & finally

The `try`, `except`, and `finally` structure provides a mechanism to manage and handle exceptions that arise during the execution of a designated block of code.

```
>>> try:
...     print("Attempting the calculation...")
...     result = 10 / 0
...
...     print(f"The result is {result}") # This line will be skipped because the error
 ↪  happens above
...
... except ZeroDivisionError:
...     print("Error: You cannot divide a number by zero!")
... finally:
...     print("Cleanup: The division operation has finished.")
...
Attempting the calculation...
Error: You cannot divide a number by zero!
Cleanup: The division operation has finished.
```

Code 71: Exception handling with `try` / `except` / `finally` block

As demonstrated above, the code within the `try` block executes first. If a specified exception—in this case, a `ZeroDivisionError`—is raised, the runtime environment prevents premature termination and transfers control to the `except` block. The `finally` block subsequently executes regardless of whether the `try` block succeeded or failed. Consequently, the `finally` block is typically reserved for essential cleanup operations, such as terminating database connections or closing files.

If a specific exception type is not explicitly defined, a generic `except` block will default to catching all potential exceptions. This approach is generally considered **poor practice**, as it can obscure unrelated bugs

and lead to unpredictable program behavior, particularly in complex software.

```
>>> try:
...     print("Attempting the calculation...")
...     result = 10 / 0
...
...     print(f"The result is {result}") # This line will be skipped because the error happens above
...
... except: # catches ALL errors
...     print("Error: You cannot divide a number by zero!")
... finally:
...     print("Cleanup: The division operation has finished.")
...
Attempting the calculation...
Error: You cannot divide a number by zero!
Cleanup: The division operation has finished.
```

Code 72: Catching all exceptions with generic `except:` header; *poor practice*

A more robust practice involves implementing multiple `except` headers to handle distinct exception types individually. Utilizing the base `Exception` class as a fallback, paired with `as e` to capture the error object, allows the program to safely identify and log unexpected exceptions while maintaining application stability.

```
>>> try:
...     user_input = "Hello"
...     number = int(user_input)
...
... except ValueError as e:
...     print(f"The message is: {e}")
...     print(f"The error name is: {type(e).__name__}")
... except Exception as e: # This is a fallback that catches anything else and prints its name
...     print(f"An unexpected {type(e).__name__} occurred: {e}")
...
The message is: invalid literal for int() with base 10: 'Hello'
The error name is: ValueError
```

Code 73: Multiple `except` blocks

6.2 Logging

Core Concept 6.4: Logging

Logging is a systematic mechanism for tracking events, errors, and state changes that occur during software execution. While standard `print()` statements are frequently utilized for *preliminary* debugging, the built-in `logging` library provides a more robust framework suitable for larger applications. It allows developers to **categorize outputed messages by severity**.

The logging framework categorizes events into five primary severity levels, listed in increasing order of critical importance: `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`. By establishing a minimum severity threshold, the runtime environment can automatically filter out lower-level messages, ensuring that only pertinent information is captured and recorded.

```
>>> import sys
```

```
>>> import logging

>>> logging.basicConfig(level=logging.WARNING, format='%(levelname)s: %(message)s',
    stream=sys.stdout, force=True) # force=True and stream=sys.stdout is needed to output
    the logs within this pyconsole environment

>>> def divide_values(a, b):
...     # This will be suppressed because DEBUG is lower than WARNING
...     logging.debug(f"Attempting to divide {a} by {b}.")
...     if b == 0:
...         # This will be captured because ERROR is higher than WARNING
...         logging.error("Division by zero attempted.")
...         return None
...     result = a / b
...     # This will be suppressed because INFO is lower than WARNING
...     logging.info(f"Division successful. Result is {result}.")
...     return result
...
>>> logging.warning("System is operating with minimal resources.")
WARNING: System is operating with minimal resources.
>>> divide_values(10, 0)
ERROR: Division by zero attempted.
```

Code 74: `logging` module usage at `WARNING` level

To retain log records for historical reference, a destination file can be specified using the `filename` parameter within the `logging.basicConfig()` function, which will route all subsequent log entries to that designated file.

6.3 Assertion

Core Concept 6.5: Assertions

An **assertion** is a programmatic debugging aid that functions as an internal sanity check. It allows developers to explicitly declare a condition that must evaluate to true at a specific point during execution. If the condition holds true, the program proceeds normally without interruption. However, if the condition evaluates to false, the runtime environment immediately halts execution and raises an `AssertionError`.

In Python, assertions are implemented utilizing the `assert` keyword, which evaluates a boolean expression and can optionally include a diagnostic error message.

```
>>> def apply_discount(original_price, discount_rate):
...     # Assert that the parameters fall within logically valid bounds
...     assert original_price > 0, "The original price must be strictly positive."
...     assert 0 <= discount_rate <= 1, "The discount rate must be between 0 and 1."
...     return original_price * (1 - discount_rate)
...
>>> final_price = apply_discount(100.0, 0.2) # This will execute successfully
>>> print(f"Discounted price: {final_price}")
Discounted price: 80.0

>>> apply_discount(-50.0, 0.2) # This will immediately trigger an AssertionError
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in apply_discount
AssertionError: The original price must be strictly positive.
```

Code 75: Internal logic verification through `assert` keyword

In practice, assertions are designed to **verify internal logic** and identify states that the developer considers computationally impossible, primarily used as tools for development and testing phases.

7 File Input/Output and Data Handling

Core Concept 7.1: Files

A **file** is a continuous, one-dimensional **stream** of data—either plain text characters or raw binary bytes—that an application can sequentially read from or write to during execution.

Core Concept 7.2: File Input/Output

To interact with a file, a program must request a **file object** from the operating system. This object acts as a temporary communication bridge between the application's volatile memory and the persistent storage drive. The fundamental lifecycle of programmatic File I/O strictly adheres to three sequential phases.

(a) **Open:** Establishing the stream connection and declaring the intended access mode.

- `'r'` / **read**: Opens a file exclusively for reading, positioning the stream at the beginning of the document. If the specified file does not exist, the runtime raises a `FileNotFoundError`.
- `'w'` / **write**: If the file already exists, its **existing contents are immediately truncated** (*completely overwritten*) before new data is introduced. If it does not exist, a new file is generated.
- `'a'` / **append**: Opens a file for writing but **preserves existing data** by positioning the stream at the very end of the document. Any newly transmitted data is appended to the conclusion of the file. If the file does not exist, a new one is created.
- `'x'` / **exclusive creation**: Opens a file strictly for writing, functioning as a safeguard against data loss. **The file must not exist prior to execution**; if it does, a `FileExistsError` is raised, intentionally aborting the operation.

If working with binary files, simply append `'b'` to the end: `'rb'`, `'wb'`, `'ab'`, `'xb'`

(b) **Process:** Transmitting data through the stream via read or write operations.

(c) **Close:** Severing the connection to flush internal memory.

Modern Python development heavily relies on the `with` statement, which acts as a context manager. This structure guarantees that the file stream is **automatically and safely closed once the nested block of code finishes execution**, even if an unexpected exception is raised during the processing phase.

```
>>> with open("sensor_data.txt", "w") as file_stream:
...     _ = file_stream.write("timestamp: 001, status: active\n")
...     _ = file_stream.write("timestamp: 002, status: active\n")
...     # file stream is automatically closed upon exiting this block
...
>>> with open("sensor_data.txt", "r") as file_stream:
...     content = file_stream.read()
...     print(content)
...
```

```
timestamp: 001, status: active
timestamp: 002, status: active
```

Code 76: File opening and closing through `with` statement

Core Concept 7.3: Serialization and Pickling

In data handling, **serialization** is the process of converting an in-memory object into a byte stream suitable for storage or transfer. In Python, this implemented via the `pickle` module. The inverse operation—reconstructing a Python object from a byte stream—is known as **deserialization** (*or unpickling*).

Pickling is highly advantageous when working with complex, custom data structures that cannot be easily represented in standard, human-readable text formats like JSON or CSV.

The `pickle` API primarily relies on two functions for file interactions:

- `pickle.dump(obj, file)` serializes the target object and writes the resulting bytes directly to an open binary file stream.
- `pickle.load(file)` reads a binary file stream containing a serialized byte string and reconstructs the original Python object.

```
>>> import pickle

>>> session_data = { # complex data structure
...     "user_id": 1042,
...     "permissions": ["admin", "developer"],
...     "matrix_weights": (0.85, 0.12, 0.94)
... }

>>> with open("session.pkl", "wb") as binary_stream: # serializing data
...     pickle.dump(session_data, binary_stream)
...
>>> with open("session.pkl", "rb") as binary_stream: # deserializing data
...     retrieved_data = pickle.load(binary_stream)
...
>>> print(f"Data Type: {type(retrieved_data)}")
Data Type: <class 'dict'>
>>> print(f"Content: {retrieved_data}")
Content: {'user_id': 1042, 'permissions': ['admin', 'developer'], 'matrix_weights': (0.85,
 ↪  0.12, 0.94)}
```

Code 77: Pickle object serialization through `pickle` module commands `.dump()` and `.load()`

It is important to keep in mind that deserializing a corrupted or maliciously constructed byte stream can trigger arbitrary code execution within the runtime environment. Consequently, objects should never be unpickled from untrusted or unauthenticated external sources.

8 Regular Expressions

Core Concept 8.1:

9 Threads

10 Networking

11 Modules & Packages

Library is the umbrella term

modules = flat file library packages = directory structure library

12 Python Ecosystem

Core Concept 12.1: Libraries

12.1 Standard Library

modules (built in) random https://docs.python.org/3/library/random.html math https://docs.python.org/3/library/math.html datetime https://docs.python.org/3/library/datetime.html time https://docs.python.org/3/library/time.html

os https://docs.python.org/3/library/os.html sys https://docs.python.org/3/library/sys.html abc https://docs.python.org/3/library re https://docs.python.org/3/library/re.html Full Library https://docs.python.org/3/py-modindex.html

12.2 Package Management

pip and PyPI conda

12.3 Third-Party Libraries

packages requests https://requests.readthedocs.io/en/latest/ seaborn https://seaborn.pydata.org/generated/seaborn.objects.Plot.ht IPython.display https://ipython.readthedocs.io/en/stable/api/generated/IPython.display.html scikit https://scikit-learn.org/1.5/api/index.html statsmodels https://www.statsmodels.org/stable/api.html scipy https://docs.scipy.org/doc/scipy/refe

12.3.1 Data Science

numpy https://numpy.org/doc/stable/reference/ pandas https://pandas.pydata.org/docs/reference/index.html matplotlib https://matplotlib.org/stable/api/index.html

12.3.2 placeholder

13 Other

del input() print sep end f string formatting